PROGRAMMING & CS FUNDAMENTALS

Stacks & Queues Visual Guide

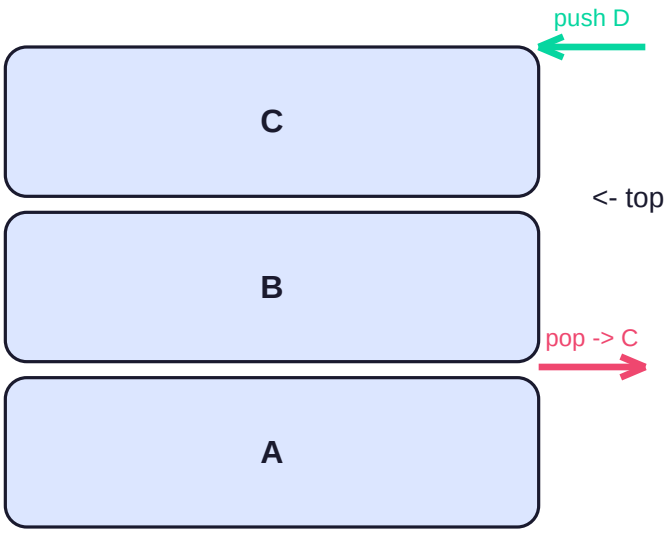
LIFO and FIFO —
two disciplines that run parsers, schedulers, and search.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Stack: Last In, First Out

Push on top, pop from top — like a stack of plates.



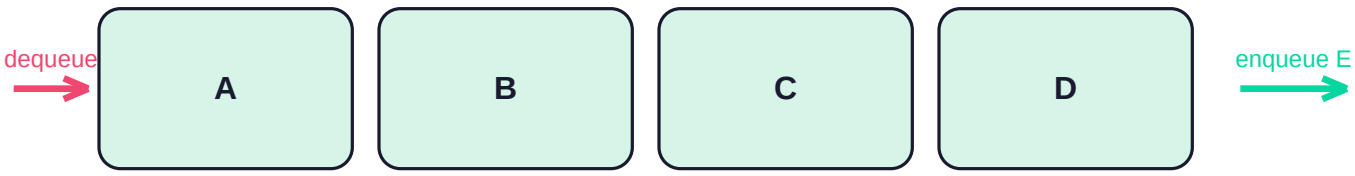

```
push(x): stack.append(x)   # O(1)
pop():   return stack.pop()  # O(1), LIFO order
peek():  stack[-1]           # look, don't touch
```

- The call stack IS a stack: each call pushes a frame, return pops it.
- Undo (Ctrl+Z), back-button history, expression parsing — all stacks.
- Overflow: pushing past capacity (or infinite recursion) crashes the party.

STEP 2/6

Queue: First In, First Out

Enqueue at the rear, dequeue from the front — a checkout line.



front ->

<- rear

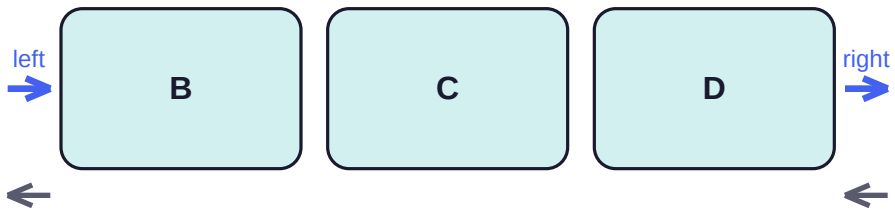
```
from collections import deque
q = deque(); q.append(x)   # enqueue O(1)
q.popleft()               # dequeue O(1), FIFO order
```

- Never popleft() a Python list — it's O(n) shifts. deque is the real queue.
- BFS, print spoolers, task schedulers, message brokers: all FIFO at heart.
- Circular buffer: fixed-size queue in an array with head/tail wrapping via modulo.

STEP 3/6

Deque: Double-Ended Queue

Push and pop from BOTH ends — stack and queue in one.




```
dq.appendleft(x); dq.append(x)      # push either end O(1)
dq.popleft();     dq.pop()         # pop either end O(1)
```

- Sliding-window maximum, palindrome checks, work-stealing schedulers.
- A deque used from one end only IS a stack; from opposite ends, a queue.
- Implemented as a doubly-linked list of blocks — O(1) both ends, cache-friendly chunks.

STEP 4/6

Pattern: Monotonic Stack

'Next greater element' in O(n) — each element pushed and popped once.

nums = [2, 1, 2, 4, 3] -> next greater = [4, 2, 4, -1, -1]

```
st = []                       # decreasing stack of indices
for i, x in enumerate(nums):
    while st and nums[st[-1]] < x:
        ans[st.pop()] = x     # x is their 'next greater'
    st.append(i)
```

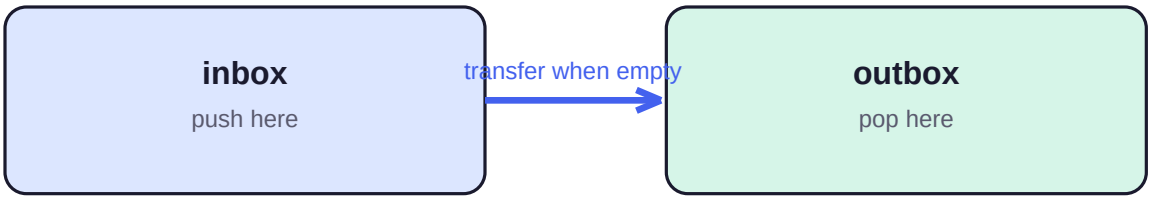
- Invariant: the stack stays monotonic (decreasing) — violators get resolved immediately.
- Each index enters and leaves once: O(n) total, despite the nested loop.
- Siblings: daily temperatures, largest rectangle in histogram, trapping rain water.

The 'while inside for looks O(n^2)' trap: count total pushes/pops, not loop nesting. Amortized thinking again.

STEP 5/6

Queue via Two Stacks

Amortized O(1) FIFO from LIFO parts — a favorite interview build.



enqueue: push to inbox. dequeue: if outbox empty, pour inbox into outbox (reverses order), then pop.

```
def dequeue(self):
    if not self.out:                    # refill rarely
        while self.inbox: self.out.append(self.inbox.pop())
    return self.out.pop()                # O(1) amortized
```

Each element moves inbox->outbox exactly once: n dequeues cost O(n) total. The reverse trick (stack via queues) is the mirror image.

STEP 6/6

Where They Show Up

Recognize the shape, reach for the structure.

Undo / redo

stack of actions

Browser back button

stack of pages

BFS graph search

queue of frontier nodes

Task scheduler

queue of ready jobs

Expression evaluation

two stacks: operands + operators

Sliding window max

monotonic deque

STEP ✓I✓

Key Takeaways

Stacks & Queues — Remember

- Stack = LIFO (push/pop O(1)); queue = FIFO (enqueue/dequeue O(1)) — use deque, not list, for queues.
- The call stack, undo history, and expression parsing are all stacks in disguise.
- Monotonic stack solves 'next greater' patterns in O(n) — each element pushed/popped once.
- Two stacks build an amortized-O(1) queue; the transfer reverses order exactly once per element.
- Deque gives O(1) at both ends — stack and queue unified.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev